PYTHON-ACTIVITY BASED LEARNING

NAME : Budda charishma

REGISTRATION NUMBER : 2600382

SECTION : CSD-1

CASE-1 UNIVERSITY SCHOLARSHIP ALLOCATION

PROBLEM STATEMENT : Design a system that evaluates applicants using aggregate marks, subject-wise minimum scores, attendance, and income criteria, ranks eligible students, and resolves ties using predefined rules.

COMPUTATIONAL THINKING :

Decomposition: marks, attendance, income checks.

Pattern: same criteria for all.

Abstraction: only tuple (name, marks tuple, att, inc).

Algorithm: unpack -> filter -> sort.

ALGORITHM : Selection of Eligible Applicants

Step 1: Start.

Step 2: Input the applicants as a nested list of tuples containing name, marks, attendance, and income.

Step 3: For each applicant, extract the name, marks, attendance, and income.

Step 4: Check whether all subject marks are at least 35, attendance is at least 75%, income is at most ₹2,00,000, and aggregate marks are at least 60%.

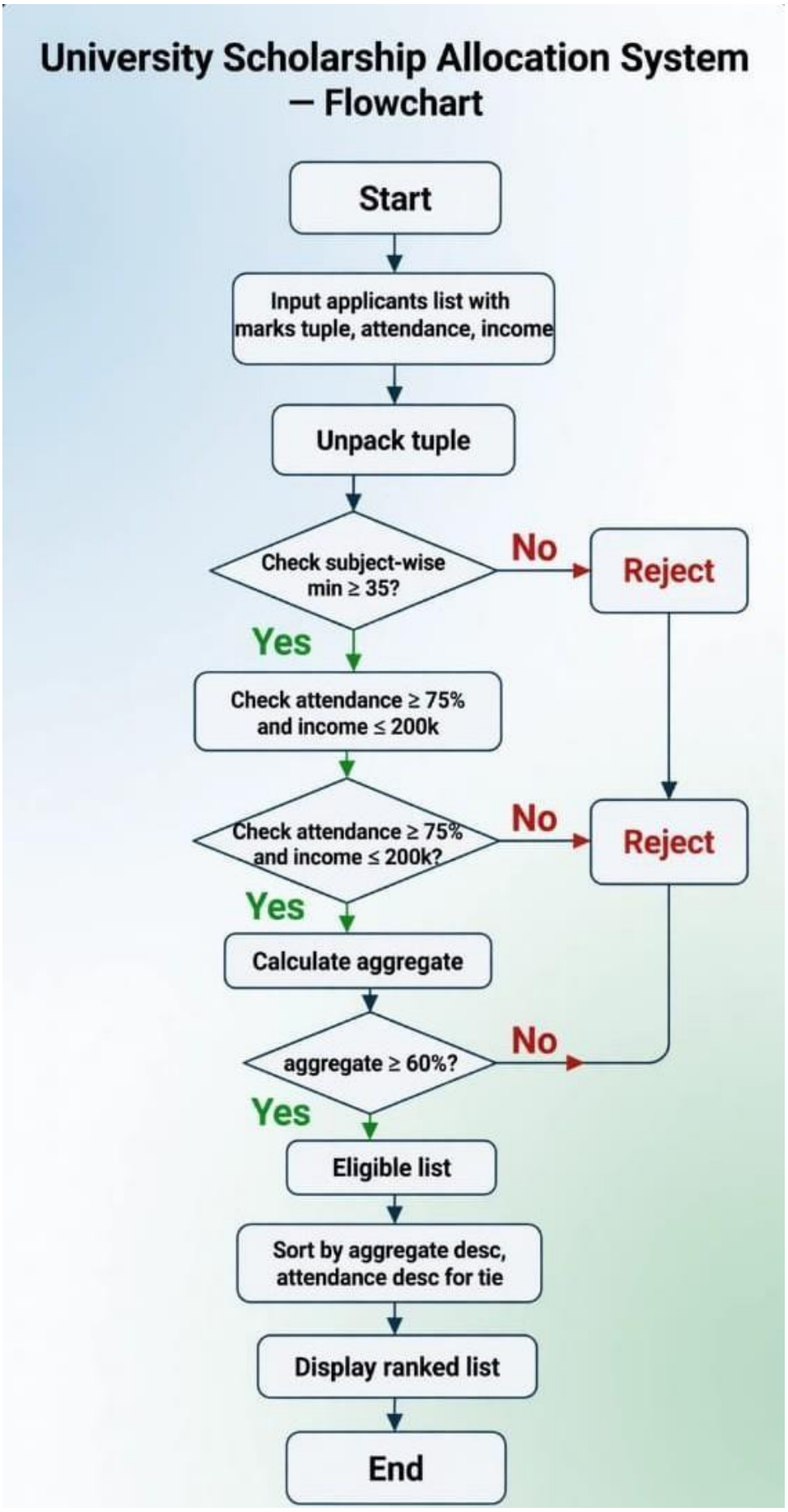
Step 5: If all conditions are satisfied, add the applicant to the eligible list.

Step 6: Sort the eligible applicants in descending order of aggregate marks. If aggregate marks are equal, sort by attendance in descending order.

Step 7: Display the list of eligible applicants.

Step 8: End.

FLOWCHART :


University Scholarship Allocation System — Flowchart
Start
Input applicants list with marks tuple, attendance, income
Unpack tuple
Check subject-wise min ≥ 35?
No
Reject
Yes
Check attendance ≥ 75% and income ≤ 200k
Check attendance ≥ 75% and income ≤ 200k?
No
Reject
Yes
Calculate aggregate
aggregate ≥ 60%?
No
Yes
Eligible list
Sort by aggregate desc, attendance desc for tie
Display ranked list
End

PSEUDOCODE :

eligible=[]

FOR (name, marks_tuple, att, inc) IN applicants:

s1,s2,s3 = marks_tuple

agg = (s1+s2+s3)/3

IF s1<35 OR s2<35 OR s3<35: CONTINUE

IF att<75 OR inc>200000: CONTINUE

IF agg<60: CONTINUE

eligible.ADD(name, agg, att)

SORT eligible BY agg DESC, att DESC

PRINT Rank

VARIABLES :

name, marks_tuple , attendance, income, s1,s2,s3, aggregate, eligible, rank.

DATA TYPES :

int (marks, att), float (agg), str (name), tuple (marks_tuple, applicant), list (applicants, eligible).

OPERATORS :

Arithmetic: + /

Relational: >= <= < >

Logical: AND, OR

Assignment: =

CONDITIONAL STATEMENTS :

if s1>=35 and s2>=35 and s3>=35: pass

if att>=75 and inc<=200000: pass

if agg>=60: eligible

LOOPS:

for applicant in applicants:

for i, (n,a,at) in enumerate(eligible,1):

STRINGS :

f"{i}. {name} - {agg:.1f}%" - f-string

name.upper() len(name)

PATTERN GENERATION :

print("_"_20) rank header

for i in range(1,4): print("_"_i) decorative

LISTS :

applicants = [] eligible = [] eligible.append()

NESTED LISTS :

applicants = [("Raju",(78,82,75),85,150000), ("Sita",(90,92,88),92,180000)] - list of tuples

TUPLES :

(78,82,75) marks immutable - change cheyodu

TUPLE OPERATIONS :

applicants[0] indexing, marks_Tuple[0] first subject, len(applicants), sort(key=lambda x:x[1])

UNPACKING :

name, marks_tuple, att, inc = applicant - main

s1,s2,s3 = marks_tuple - nested unpacking

n,a,at = eligible[0] – result

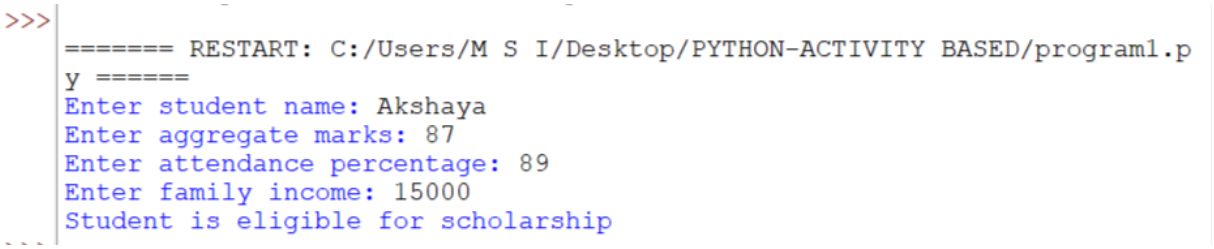
INPUT(CODE):

```
name = input("Enter student name: ")
marks = float(input("Enter aggregate marks: "))
attendance = float(input("Enter attendance percentage: "))
```

```
income = float(input("Enter family income: "))


if marks >= 80 and attendance >= 75 and income <= 200000:
    print("Student is eligible for scholarship")
else:
    print("Student is not eligible")
```

OUTPUT:



CASE-2 PROGRRESSIVE ELECTRICITY BILLING

PROBLEM STATEMENT :

Calculate household bills using multiple consumption slabs, fixed charges, surcharges, and rebates, then compare bills across several months and identify unusual increases.

COMPUTATIONAL THINKING :

- Decomposition: Slab calc + Fixed + Rebate/Surcharge + Unusual detection
- Pattern Recognition: 0-100=Rs3, 101-200=Rs5, 200+=Rs7 slab repeat
- Abstraction: Month colour vaddu - only month, units, bill kavali
- Algorithm Design: Unpack -> Slab formula -> Adjust -> Compare

ALGORITHM :

Step 1: START

Step 2: INPUT months_data list of tuples (month, units)

Step 3: SET bills=[]

Step 4: FOR each IN months_data:

UNPACK month, units

IF units<=100: bill=units_3 ELIF <=200: bill=300+(units-100)_5 ELSE bill=800+(units-200)_7

bill+=100 fixed

IF units<100: bill_=0.95 rebate IF units>500: bill_=1.10 surcharge

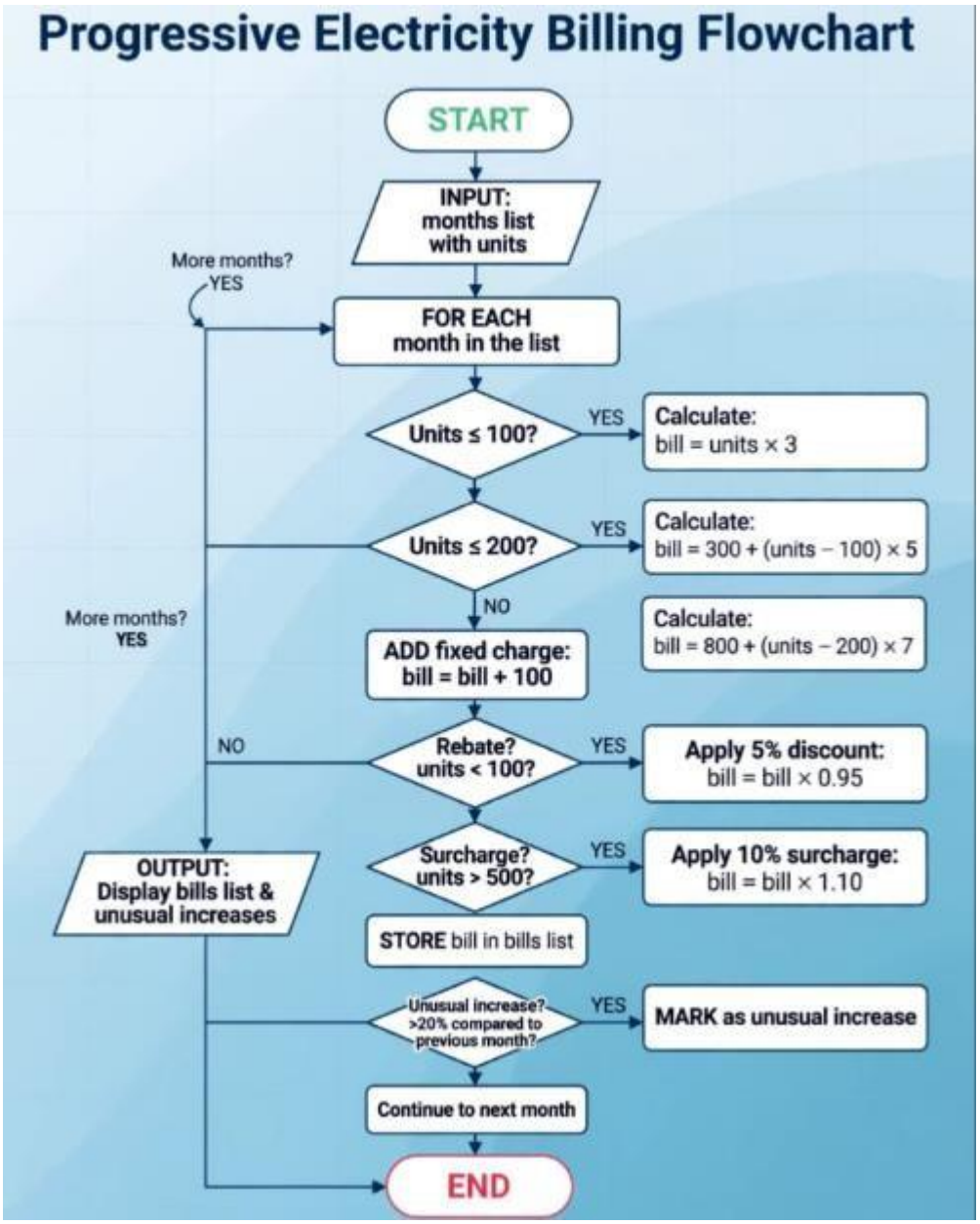
APPEND (month, units, bill)

Step 5: FOR i=1 TO len(bills)-1: IF http://bills.bill > bills[i-1].bill_1.2 PRINT Unusual

Step 6: DISPLAY

Step 7: END[i]

FLOWCHART :



PSEUDOCODE:

```
bills=[]
FOR (month, units) IN months_data:
  IF units<=100: bill=units*3
  ELIF units<=200: bill=300+(units-100)*5
  ELSE: bill=800+(units-200)*7
  bill+=100
  IF units<100: bill*=0.95
  bills.ADD(month, units, bill)
```

FOR i=1..len(bills)-1:

IF bills[i].bill > bills[i-1].bill*1.2: PRINT Unusual

VARIABLES :

month, units, bill, fixed_charge, rebate, surcharge, bills

DATA TYPES :

int (units), float (bill), str (month), tuple, list

OPERATORS : + - _ / <= >

CONDITIONAL :

if units<=100 elif <=200 else, if units<100 rebate

LOOPS :

for month,units in months_data, for i in range(1,len(bills))

STRINGS :

f"{month} Bill Rs.{bill}" f-string

PATTERN :

print("="_20) bill separator

LISTS :

bills=[] append

NESTED LISTS :

months=[("Jan",120),("Feb",180)]

TUPLES :

("Jan",120) immutable month data

TUPLE OPERATIONS :

months, months, len()[0]

UNPACKING :

month, units = data AND prev_m, prev_u, prev_b = bills[i-1]

INPUT(CODE) :

```
units = int(input("Enter electricity units: "))


if units <= 100:
    bill = units * 2
elif units <= 200:
    bill = 100 * 2 + (units - 100) * 3
else:
    bill = 100 * 2 + 100 * 3 + (units - 200) * 5


bill = bill + 50
print("Electricity Bill =", bill)
```

OUTPUT :

```
>>
=== RESTART: C:/Users/M S I/Desktop/PYTHON-ACTIVITY BASED/program2.py ==
Enter electricity units: 36
Electricity Bill = 122
>>
```

CASE -3 HOSPITAL BILLING AND PATIENT CLASSIFICATION

PROBLEM STATEMENT : Process consultation, diagnostic, and treatment charges, apply conditional concessions, validate billing constraints, and classify patients according to a supplied payment policy.

COMPUTATIONAL THINKING :

- Decomposition: Total calc + Concession + Classification + Validation

- Pattern: Age>60=10%, BPL=20% discount repeat

- Abstraction: Name, charges, age, bpl only

- Algorithm: Unpack -> Total -> Validate -> Discount -> Classify

3. ALGORITHM:

Step 1: Start.

Step 2: Input the patients as a tuple list containing name, consultation fee (c), diagnostic fee (d), treatment fee (t), age, and BPL status.

Step 3: For each patient, extract the name, consultation fee, diagnostic fee, treatment fee, age, and BPL status.

Step 4: Calculate the total medical bill by adding the consultation fee, diagnostic fee, and treatment fee.

Step 5: If the total bill is less than or equal to zero, skip the invalid patient and continue to the next patient.

Step 6: Initialize the concession amount to zero.

Step 7: If the patient's age is greater than 60 years, calculate a 10% concession on the total bill and add it to the concession amount.

Step 8: If the patient's BPL status is "Yes", calculate a 20% concession on the total bill and add it to the concession amount.

Step 9: Calculate the payable amount by subtracting the total concession from the total bill.

Step 10: If the payable amount is less than ₹5,000, classify the bill as "Low".

Step 11: Else, if the payable amount is less than or equal to ₹15,000, classify the bill as "Medium".

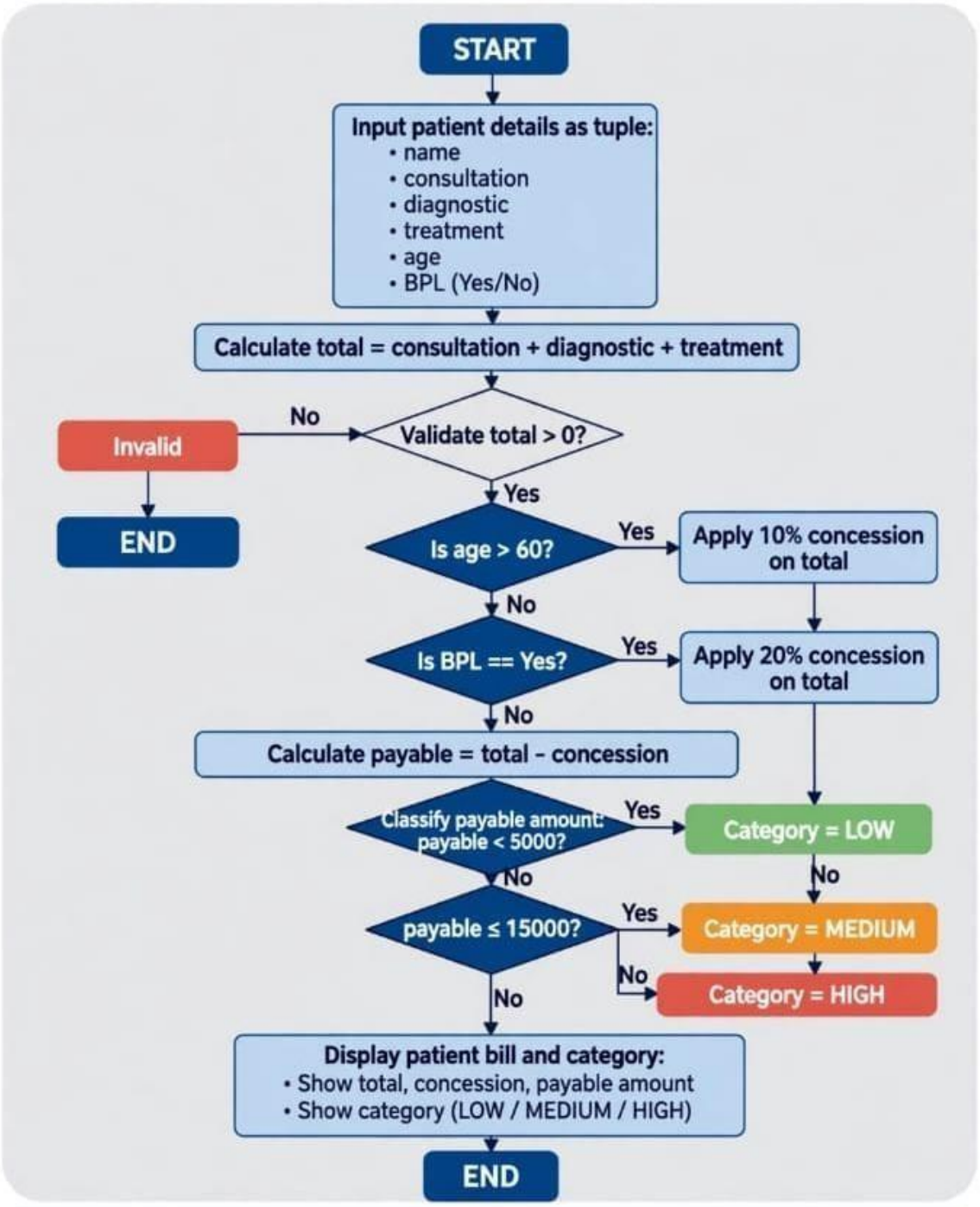
Step 12: Otherwise, classify the bill as "High".

Step 13: Display the patient's name, total bill, concession amount, payable amount, and bill category.

Step 14: End.

FLOWCHART :

Hospital Billing and Patient Classification Flowchart



PSEUDOCODE :

bills=[]

FOR (month, units) IN months_data:

IF units<=100: bill=units*3

ELIF units<=200: bill=300+(units-100)*5

ELSE: bill=800+(units-200)*7

bill+=100

IF units<100: bill*=0.95

bills.ADD(month, units, bill)

Syllabus :

Variables: name, consult, diag, treat, age, bpl, total, payable, cat

Data Types: int, float, str, tuple, list

Operators: + _ < >

Conditional: if age>60, if bpl=="Yes", if payable<5000

Loops: for record in patients

Strings: f"{name} | {cat}"

Pattern: print("-"_15)

Lists: patients list

Nested: [("Raju",5000,3000,2000,65,"No")]

Tuples: immutable patient

Tuple Op: patients, len()

Unpacking: name,c,d,t,age,bpl = record[0]

INPUT(CODE) :

```
consultation = float(input("Enter consultation charge: "))
diagnostic = float(input("Enter diagnostic charge: "))
treatment = float(input("Enter treatment charge: "))

total = consultation + diagnostic + treatment

if total >= 5000:
    discount = total * 0.10
```

else:

discount = 0

final_bill = total - discount

print("Total Bill =", total)

print("Discount =", discount)

print("Final Bill =", final_bill)

OUTPUT :

```
=== RESTART: C:/Users/M S I/Desktop/PYTHON-ACTIVITY BASED/program3.py ==
Enter consultation charge: 500
Enter diagnostic charge: 800
Enter treatment charge: 300
Total Bill = 1600.0
Discount = 0
Final Bill = 1600.0
```

CASE-4 EMPLOYEE PAYROLL AND DEDUCTION ANALYSIS

PROBLEM : Calculate gross salary from basic pay, allowances, overtime, and incentives; apply progressive deductions and generate a formatted salary statement.

COMPUTATIONAL THINKING :

Decomposition: Allowances + Gross + Deduction + Slip

Pattern: DA=10% HRA=15% OT=200/hr fixed repeat

Abstraction: basic, ot_hrs, incentive only

Algorithm: Unpack -> da,hra,ot -> gross -> ded -> net -> slip

ALGORITHM :

Step 1: Start.

Step 2: Input the employees list containing name, basic salary, overtime hours, and incentives.

Step 3: For each employee, extract the name, basic salary, overtime hours, and incentives.

Step 4: Calculate Dearness Allowance (DA) as 10% of the basic salary.

Step 5: Calculate House Rent Allowance (HRA) as 15% of the basic salary.

Step 6: Calculate overtime pay by multiplying overtime hours by ₹200.

Step 7: Calculate gross salary by adding basic salary, DA, HRA, overtime pay, and incentives.

Step 8: If the gross salary is less than or equal to ₹20,000, calculate the deduction as 5% of the gross salary.

Step 9: Else, if the gross salary is less than or equal to ₹40,000, calculate the deduction as 10% of the gross salary.

Step 10: Otherwise, calculate the deduction as 15% of the gross salary.

Step 11: Calculate the net salary by subtracting the deduction from the gross salary.

Step 12: Print the formatted salary slip containing employee details, gross salary, deduction, and net salary.

Step 13: End.

4. FLOWCHART:

Syllabus:

Variables: name, consult, diag, treat, age, bpl, total, payable, cat

Data Types: int, float, str, tuple, list

Operators: + _ < >

Conditional: if age>60, if bpl=="Yes", if payable<5000

Loops: for record in patients

strings f"Slip {name}",

pattern "-"*20, lists,

nested lists [("Anu",20000,10,2000)],

Tuples: immutable patient

Tuple Op: patients, len()

Unpacking : name,basic,ot,inc = emp

INPUT(CODE):

```
basic = float(input("Enter basic salary: "))
allowance = float(input("Enter allowance: "))
overtime = float(input("Enter overtime amount: "))
incentive = float(input("Enter incentive: "))


gross = basic + allowance + overtime + incentive
deduction = gross * 0.10
net_salary = gross - deduction


print("Gross Salary =", gross)
print("Deduction =", deduction)
print("Net Salary =", net_salary)
```

OUTPUT :

```
>>>
=== RESTART: C:/Users/M S I/Desktop/PYTHON-ACTIVITY BASED/program4.py ==
Enter basic salary: 25000
Enter allowance: 600
Enter overtime amount: 700
Enter incentive: 200
Gross Salary = 26500.0
Deduction = 2650.0
Net Salary = 23850.0
>>>
```

CASE-5 MULTI-PLAN TELECOMMUNICATIONS COST OPTIMIZATION

PROBLEM STATEMENT :

Design a system that evaluates multiple mobile plans based on usage, validity, and budget constraints, calculates excess charges, and identifies the most economical eligible plan.

COMPUTATIONAL THINKING :

- **Decomposition: Usage check, validity check, excess charge calculation, budget check.**
- **Pattern Recognition: Prathi mobile plan ki same logic apply avuthundi.**
- **Abstraction: Plan name, data limit, used data, validity, base cost, budget matrame consider chestham.**
- **Algorithm Design: Input plans → Calculate excess charges → Calculate total cost → Filter eligible plans → Select lowest-cost plan.**

ALGORITHM :

Step 1: START

Step 2: INPUT mobile plans as a list of tuples (name, data_limit, used_data, validity, base_cost).

Step 3: INPUT required validity and budget.

Step 4: SET eligible = [].

Step 5: FOR each plan in plans:

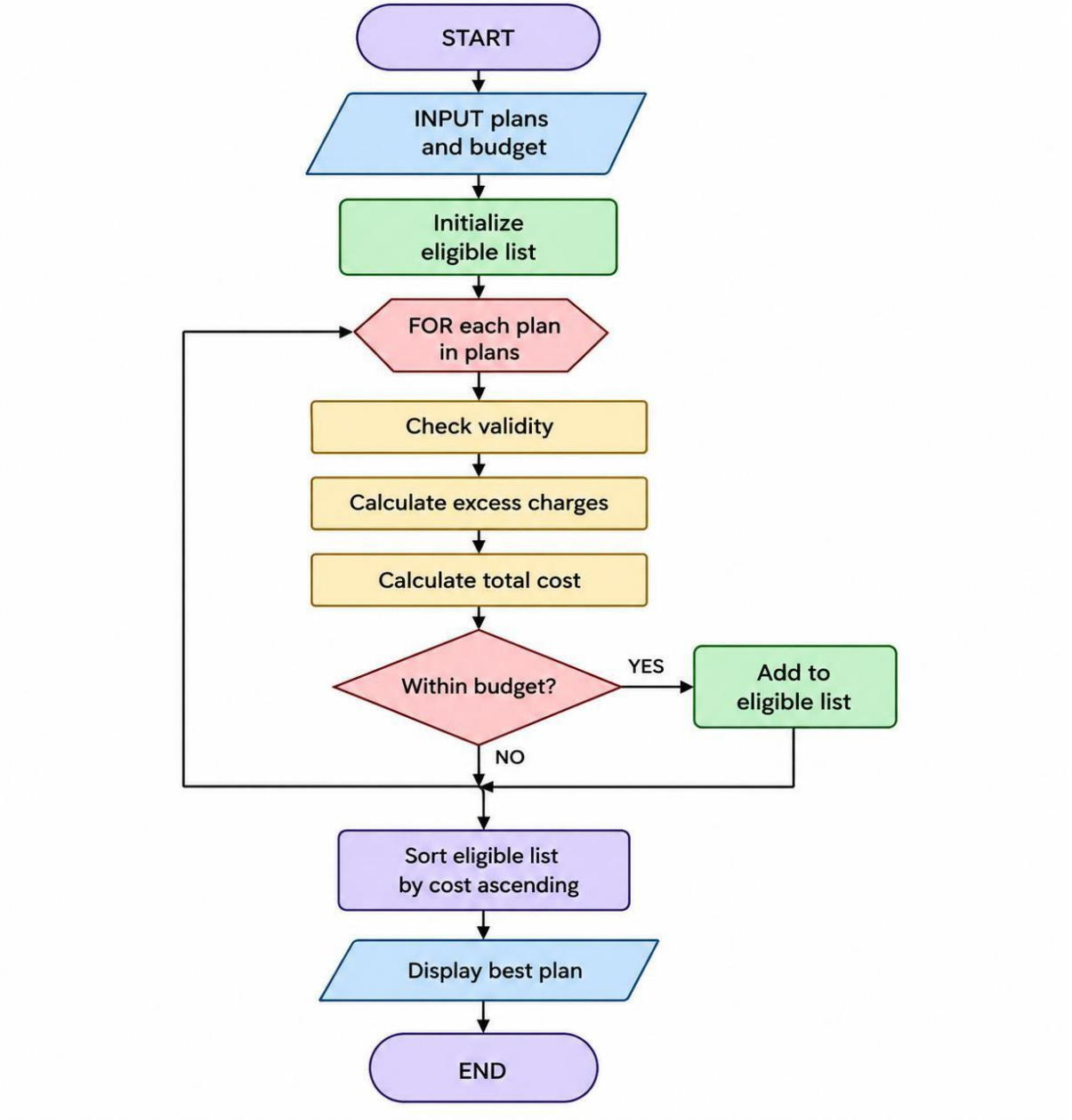
- **UNPACK name, data_limit, used_data, validity, base_cost.**
- **IF validity is less than required validity, CONTINUE.**
- **IF used_data is greater than data_limit, CALCULATE excess charges.**
- **CALCULATE total_cost = base_cost + excess charges.**
- **IF total_cost is within budget, APPEND plan details to eligible.**

Step 6: SORT eligible plans by total cost in ascending order.

Step 7: DISPLAY the most economical eligible plan.

Step 8: END.

FLOWCHART :



PSEUDOCODE :

eligible = []

FOR each plan IN plans:

UNPACK name, data_limit, used_data, validity, base_cost

IF validity < required_validity:

CONTINUE

excess = MAX(0, used_data - data_limit)

excess_charge = excess * rate

total_cost = base_cost + excess_charge

IF total_cost <= budget:

eligible.append((name, total_cost))

SORT eligible BY total_cost ASCENDING

PRINT most economical plan

VARIABLES :

name, data_limit, used_data, validity, base_cost, excess, rate, excess_charge, total_cost, budget, eligible

DATA TYPES :

- **int: data limit, used data, validity**
- **float: base cost, excess charge, total cost**
- **str: plan name**
- **tuple: individual plan**
- **list: plans, eligible**

OPERATORS :

- **Arithmetic: +, -, ***
- **Relational: >=, <=, <, >**
- **Logical: AND, OR**
- **Assignment: =**

CONDITIONAL STATEMENTS :

if validity >= required_validity: passif total_cost <= budget: pass

LOOPS :

for plan in plans: pass

Used to evaluate every mobile plan.

STRINGS :

f"{name} - Cost: ₹{total_cost:.2f}"name.upper()len(name)

PATTERN GENERATION :

print("_" * 30)for i in range(1, 4): print("*" * i)

LISTS :

plans = []eligible = []eligible.append()

NESTED LISTS :

plans = [("Plan A", 2, 3, 28, 199), ("Plan B", 3, 3, 56, 299)]

TUPLES :

("Plan A", 2, 3, 28, 199) stores one mobile plan. Tuples are immutable.

TUPLE OPERATIONS :

plans[0]plan[0]len(plans)

UNPACKING :

name, data_limit, used_data, validity, base_cost = plan

INPUT(CODE) :

```
usage = float(input("Enter monthly data usage in GB: "))

plan1 = 199
plan2 = 299
plan3 = 399

if usage <= 2:
    print("Best Plan: Plan 1")
    print("Cost =", plan1)
elif usage <= 5:
    print("Best Plan: Plan 2")
    print("Cost =", plan2)
```

```
else:
    print("Best Plan: Plan 3")
    print("Cost =", plan3)
```

OUTPUT :

```
>>>
=== RESTART: C:/Users/M S I/Desktop/PYTHON-ACTIVITY BASED/program5.py ==
Enter monthly data usage in GB: 2
Best Plan: Plan 1
Cost = 199
>>>
```

CASE-6 RAILWAY FARE AND RESERVATION CALCULATOR

PROBLEM STATEMENT :

Design a system to calculate railway fares for multiple passengers using distance slabs, travel classes, age-based concessions, reservation fees, and cancellation penalties.

COMPUTATIONAL THINKING :

- **Decomposition: Distance calculation, class fare, concession, reservation fee, cancellation penalty.**
- **Pattern Recognition: Apply the same fare rules to every passenger.**
- **Abstraction: Consider passenger name, age, distance, class, and cancellation status.**
- **Algorithm Design: Input passengers → Calculate base fare → Apply concession → Add fees → Calculate final payable amount.**

ALGORITHM :

Step 1: START.

Step 2: INPUT passenger details as a list of tuples (name, age, distance, travel_class, cancelled).

Step 3: SET fare_details = [].

Step 4: FOR each passenger:

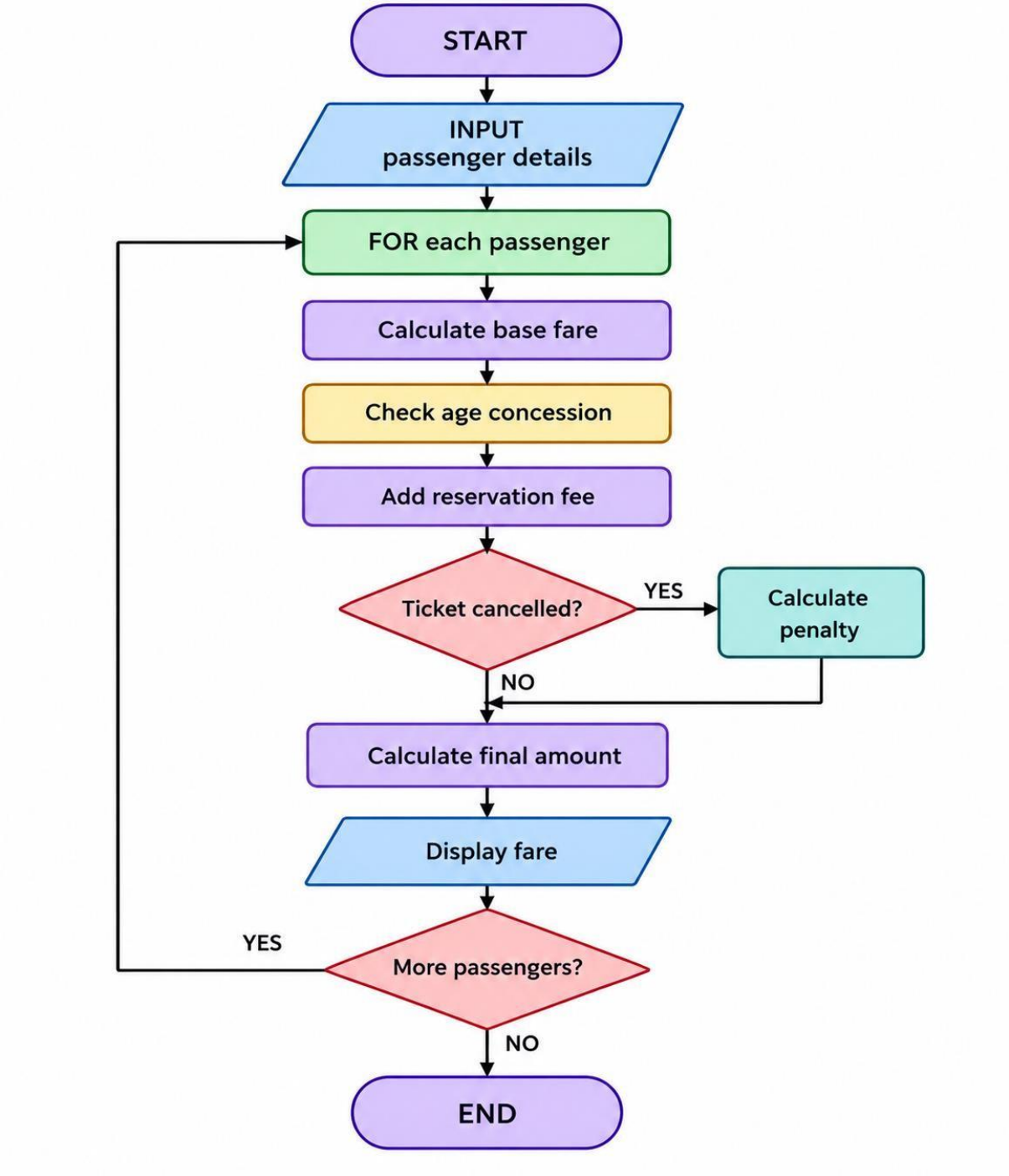
- **UNPACK name, age, distance, travel_class, cancelled.**
- **CALCULATE base fare using the distance slab and travel class.**

- **IF age qualifies for a concession, calculate the concession amount.**
- **CALCULATE reservation fee.**
- **CALCULATE final fare after concession and reservation fee.**
- **IF the ticket is cancelled, calculate the cancellation penalty.**
- **CALCULATE the final payable or refundable amount.**
- **ADD the passenger's fare details to fare_details.**

Step 5: DISPLAY the itemized fare details for all passengers.

Step 6: END.

FLOWCHART :



PSEUDOCODE :

fare_details = []

FOR passenger IN passengers:

UNPACK name, age, distance, travel_class, cancelled

base_fare = calculate_fare(distance, travel_class)

concession = calculate_concession(age, base_fare)

reservation_fee = calculate_reservation_fee(travel_class)

final_fare = base_fare - concession + reservation_fee

IF cancelled:

penalty = calculate_penalty(final_fare)

amount = final_fare - penalty

ELSE:

amount = final_fare

fare_details.append((name, final_fare, amount))

PRINT fare_details

VARIABLES :

name, age, distance, travel_class, cancelled, base_fare, concession, reservation_fee, final_fare, penalty, amount

DATA TYPES :

- **int: age, distance**
- **float: fare, concession, penalty**
- **str: name, travel class**
- **bool: cancelled**
- **tuple: passenger details**
- **list: passengers, fare_details**

OPERATORS :

Arithmetic: +, -, *, /

Relational: >=, <=, <, >

Logical: AND, OR

Assignment: =

CONDITIONAL STATEMENTS :

if age >= 60: passif cancelled: pass

LOOPS :

for passenger in passengers: pass

STRINGS :

f"{name} - Fare: ₹{final_fare:.2f}"name.upper()len(name)

PATTERN GENERATION :

print("=" * 30)for i in range(1, 4): print("*" * i)

LISTS :

passengers = []fare_details = []fare_details.append()

NESTED LISTS :

passengers = [("Ravi", 25, 200, "Sleeper", False), ("Sita", 65, 200, "AC", False)]

TUPLES :

("Ravi", 25, 200, "Sleeper", False) stores one passenger's details.

TUPLE OPERATIONS :

passengers[0]passenger[0]len(passengers)

UNPACKING :

name, age, distance, travel_class, cancelled = passenger

INPUT(CODE) :

```
distance = float(input("Enter travel distance in km: "))
age = int(input("Enter passenger age: "))

fare = distance * 2

if age >= 60:
```

```
    fare = fare * 0.80


reservation = 30

total = fare + reservation


print("Ticket Fare =", fare)

print("Reservation Fee =", reservation)

print("Total Fare =", total)
```

OUTPUT :

```
>>>
=== RESTART: C:/Users/M S I/Desktop/PYTHON-ACTIVITY BASED/program6.py ==
Enter travel distance in km: 5
Enter passenger age: 19
Ticket Fare = 10.0
Reservation Fee = 30
Total Fare = 40.0
>>>
```

CASE-7 RETAIL BILLING WITH MULTIPLE DISCOUNT RULES

PROBLEM STATEMENT :

Design a retail billing system that processes multiple products, quantities, category-specific taxes, threshold discounts, and maximum concession limits to generate an itemized invoice.

COMPUTATIONAL THINKING :

- **Decomposition: Product cost, discount, tax calculation, final bill.**
- **Pattern Recognition: Apply the same billing rules to every product.**
- **Abstraction: Consider product name, price, quantity, category, and discount rules.**
- **Algorithm Design: Input products → Calculate subtotal → Apply discounts → Limit concession → Add tax → Generate invoice.**

ALGORITHM :

Step 1: START.

Step 2: INPUT products as a list of tuples (name, price, quantity, category).

Step 3: INPUT discount thresholds and maximum concession limit.

Step 4: SET invoice = [] and subtotal = 0.

Step 5: FOR each product:

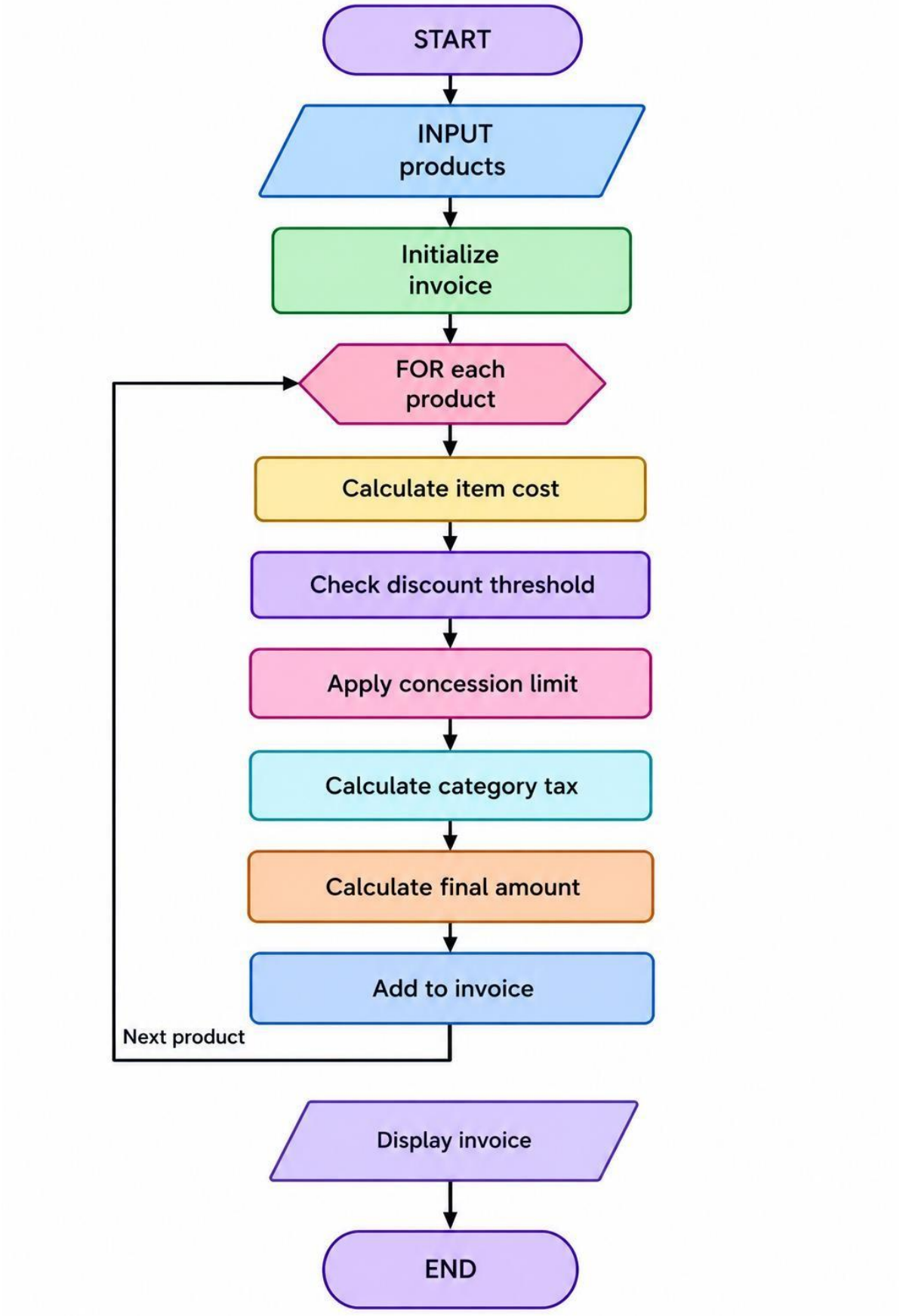
- **UNPACK name, price, quantity, category.**
- **CALCULATE item cost = price × quantity.**
- **ADD item cost to subtotal.**
- **CALCULATE discount according to the applicable threshold.**
- **APPLY the maximum concession limit.**
- **CALCULATE category-specific tax.**
- **CALCULATE the final item amount.**
- **ADD item details to invoice.**

Step 6: CALCULATE the total payable amount.

Step 7: DISPLAY the itemized invoice and total bill.

Step 8: END.

FLOWCHART :


START
INPUT products
Initialize invoice
FOR each product
Calculate item cost
Check discount threshold
Apply concession limit
Calculate category tax
Calculate final amount
Add to invoice
Next product
Display invoice
END

PSEUDOCODE :

```
invoice = []
subtotal = 0

FOR product IN products:
    UNPACK name, price, quantity, category
    item_cost = price * quantity
    subtotal = subtotal + item_cost

    discount = calculate_discount(item_cost)
    discount = MIN(discount, max_concession)
    taxable_amount = item_cost - discount
    tax = taxable_amount * tax_rate[category]
    final_amount = taxable_amount + tax

    invoice.append((name, item_cost, discount, tax, final_amount))

PRINT invoice
PRINT total payable
```

VARIABLES :

name, price, quantity, category, item_cost, subtotal, discount, max_concession, taxable_amount, tax, tax_rate, final_amount, invoice

DATA TYPES :

- **int: quantity**
- **float: price, discount, tax, final amount**
- **str: name, category**
- **tuple: product details**
- **list: products, invoice**

- **dict: category tax rates**

OPERATORS :

Arithmetic: +, -, *, /

Relational: >=, <=, <, >

Logical: AND, OR

Assignment: =

CONDITIONAL STATEMENTS :

if subtotal >= 5000: passif category == "Food": pass

LOOPS :

for product in products: pass

STRINGS :

f"{name} - Total: ₹{final_amount:.2f}"name.upper()len(name)

PATTERN GENERATION :

print("-" * 30)for i in range(1, 4): print("*" * i)

LISTS :

products = []invoice = []invoice.append()

NESTED LISTS :

products = [("Rice", 60, 5, "Food"), ("Soap", 40, 3, "Personal Care")]

TUPLES :

("Rice", 60, 5, "Food") stores product information.

TUPLE OPERATIONS :

products[0]product[0]len(products)

UNPACKING :

name, price, quantity, category = product

INPUT(CODE) :

price = float(input("Enter product price: "))

quantity = int(input("Enter quantity: "))

```
total = price * quantity


if total >= 1000:
    discount = total * 0.10
else:
    discount = 0


tax = (total - discount) * 0.05
final_bill = total - discount + tax


print("Total =", total)
print("Discount =", discount)
print("Tax =", tax)
print("Final Bill =", final_bill)
```

OUTPUT :

```
=== RESTART: C:/Users/M S I/Desktop/PYTHON-ACTIVITY BASED/program7.py ==
Enter product price: 500
Enter quantity: 100
Total = 50000.0
Discount = 5000.0
Tax = 2250.0
Final Bill = 47250.0
```

CASE-8 MANUFACTURING COST AND PROFITABILITY ANALYSIS

1. PROBLEM STATEMENT

Design a system that calculates material costs, labour charges, wastage, machine costs, and selling prices to determine whether production meets a specified profit target.

2. COMPUTATIONAL THINKING

- **Decomposition: Material cost, wastage cost, labour cost, machine cost, profit calculation.**
- **Pattern Recognition: Apply the same costing formula to every product.**
- **Abstraction: Consider product name, material quantity, material rate, labour cost, machine cost, wastage percentage, and selling price.**
- **Algorithm Design: Input production details → Calculate costs → Calculate profit → Compare with target → Display results.**

3. ALGORITHM

Step 1: START.

Step 2: INPUT production details as a list of tuples (name, material_qty, material_rate, labour_cost, machine_cost, wastage_percent, selling_price).

Step 3: INPUT the required profit target.

Step 4: SET results = [].

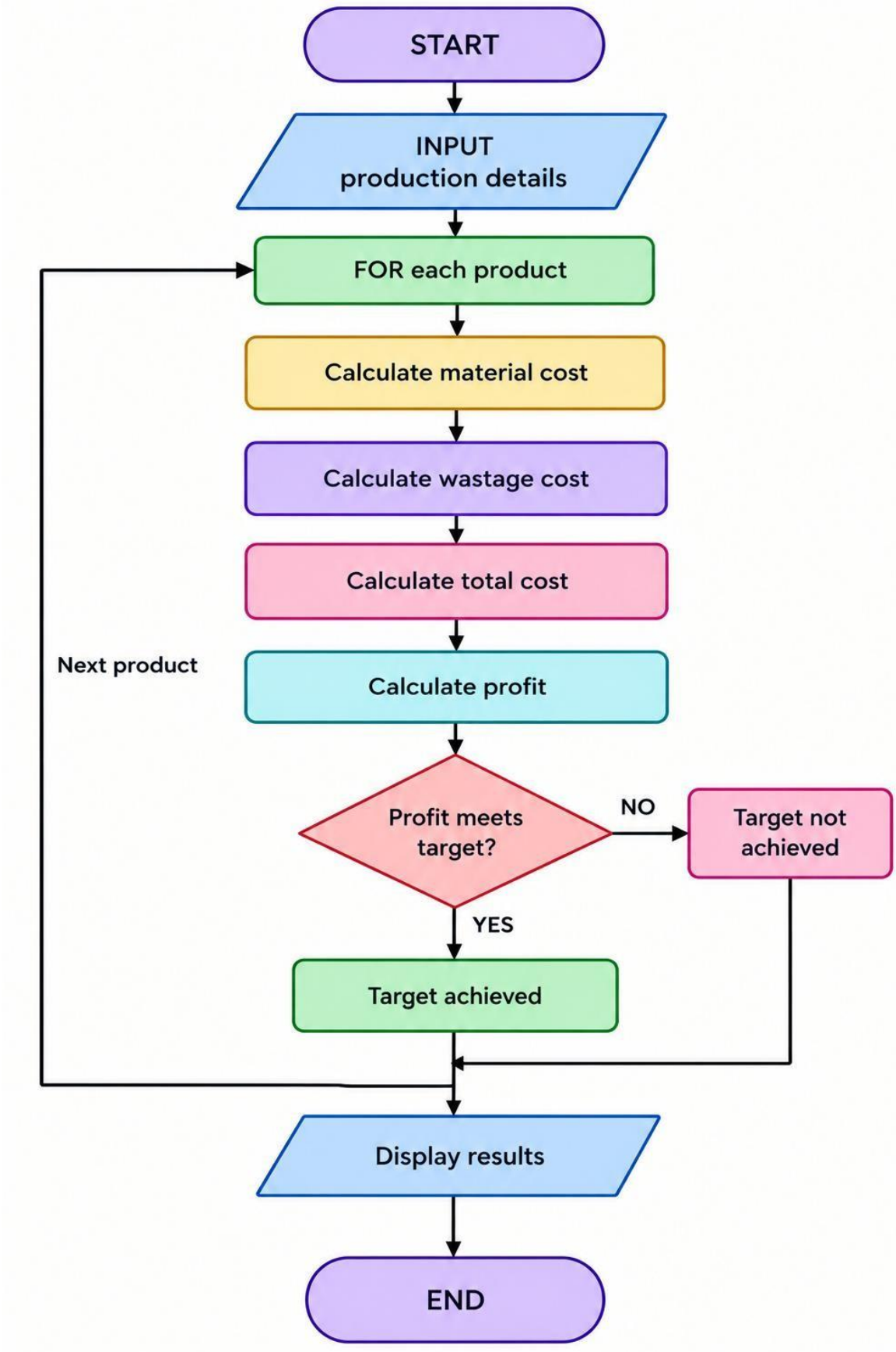
Step 5: FOR each product:

- **UNPACK all production details.**
- **CALCULATE material cost = material quantity × material rate.**
- **CALCULATE wastage cost using the wastage percentage.**
- **CALCULATE total production cost by adding material, wastage, labour, and machine costs.**
- **CALCULATE profit = selling price − total production cost.**
- **CHECK whether profit meets the target.**
- **ADD the results to the list.**

Step 6: DISPLAY production cost, profit, and target status.

Step 7: END.

4. FLOWCHART :


START
INPUT
production details
FOR each product
Calculate material cost
Calculate wastage cost
Calculate total cost
Calculate profit
Next product
Profit meets
target?
NO
Target not
achieved
YES
Target achieved
Display results
END

5. PSEUDOCODE:

```
results = []


FOR product IN products:
    UNPACK name, material_qty, material_rate,
           labour_cost, machine_cost,
           wastage_percent, selling_price


    material_cost = material_qty * material_rate
    wastage_cost = material_cost * wastage_percent / 100
    total_cost = material_cost + wastage_cost + labour_cost + machine_cost
    profit = selling_price - total_cost


    IF profit >= target_profit:
        status = "Target Achieved"
    ELSE:
        status = "Target Not Achieved"


    results.append((name, total_cost, profit, status))


PRINT results
```

6. VARIABLES

name, material_qty, material_rate, material_cost, wastage_percent, wastage_cost, labour_cost, machine_cost, total_cost, selling_price, profit, target_profit, status

7. DATA TYPES

- **int: material quantity**
- **float: costs, wastage percentage, selling price, profit**

- **str: product name, status**
- **tuple: production details**
- **list: products, results**

8. OPERATORS

Arithmetic: +, -, *, /

Relational: >=, <=, <, >

Logical: AND, OR

Assignment: =

9. CONDITIONAL STATEMENTS

if profit >= target_profit: passelse: pass

10. LOOPS

for product in products: pass

11. STRINGS

f"{name} - Profit: ₹{profit:.2f}"name.upper()len(name)

12. PATTERN GENERATION

print("=" * 30)for i in range(1, 4): print("*" * i)

13. LISTS

products = []results = []results.append()

14. NESTED LISTS

products = [("Chair", 10, 200, 500, 300, 5, 5000), ("Table", 5, 500, 800, 400, 10, 6000)]

15. TUPLES

("Chair", 10, 200, 500, 300, 5, 5000) stores production details.

16. TUPLE OPERATIONS

products[0]product[0]len(products)

17. UNPACKING

name, material_qty, material_rate, labour_cost, machine_cost, wastage_percent, selling_price = product

INPUT(CODE) :

```
material = float(input("Enter material cost: "))
labour = float(input("Enter labour cost: "))
machine = float(input("Enter machine cost: "))
selling = float(input("Enter selling price: "))

total_cost = material + labour + machine
profit = selling - total_cost

print("Total Cost =", total_cost)
print("Profit =", profit)

if profit > 0:
    print("Production is profitable")
elif profit == 0:
    print("No profit, no loss")
else:
    print("Production is at a loss")
```

OUTPUT :

```
>>>
=== RESTART: C:/Users/M S I/Desktop/PYTHON-ACTIVITY BASED/program8.py ==
Enter material cost: 1000
Enter labour cost: 500
Enter machine cost: 2000
Enter selling price: 5000
Total Cost = 3500.0
Profit = 1500.0
Production is profitable
>>>
```

CASE-9 MULTI-CURRENCY TRAVEL BUDGET ANALYSIS

1. PROBLEM STATEMENT

Design a system that converts travel expenses using supplied exchange rates, includes conversion fees, compares spending against a budget, and identifies the largest expense category.

2. COMPUTATIONAL THINKING

- **Decomposition: Currency conversion, fee calculation, budget comparison, expense analysis.**
- **Pattern Recognition: Apply the same conversion process to each expense.**
- **Abstraction: Consider expense category, amount, currency, exchange rate, and conversion fee.**
- **Algorithm Design: Input expenses → Convert amounts → Add fees → Calculate total → Compare budget → Identify largest category.**

3. ALGORITHM

Step 1: START.

Step 2: INPUT travel expenses as a list of tuples (category, amount, currency).

Step 3: INPUT exchange rates, conversion fee percentage, and travel budget.

Step 4: SET total_spending = 0 and largest_expense = 0.

Step 5: FOR each expense:

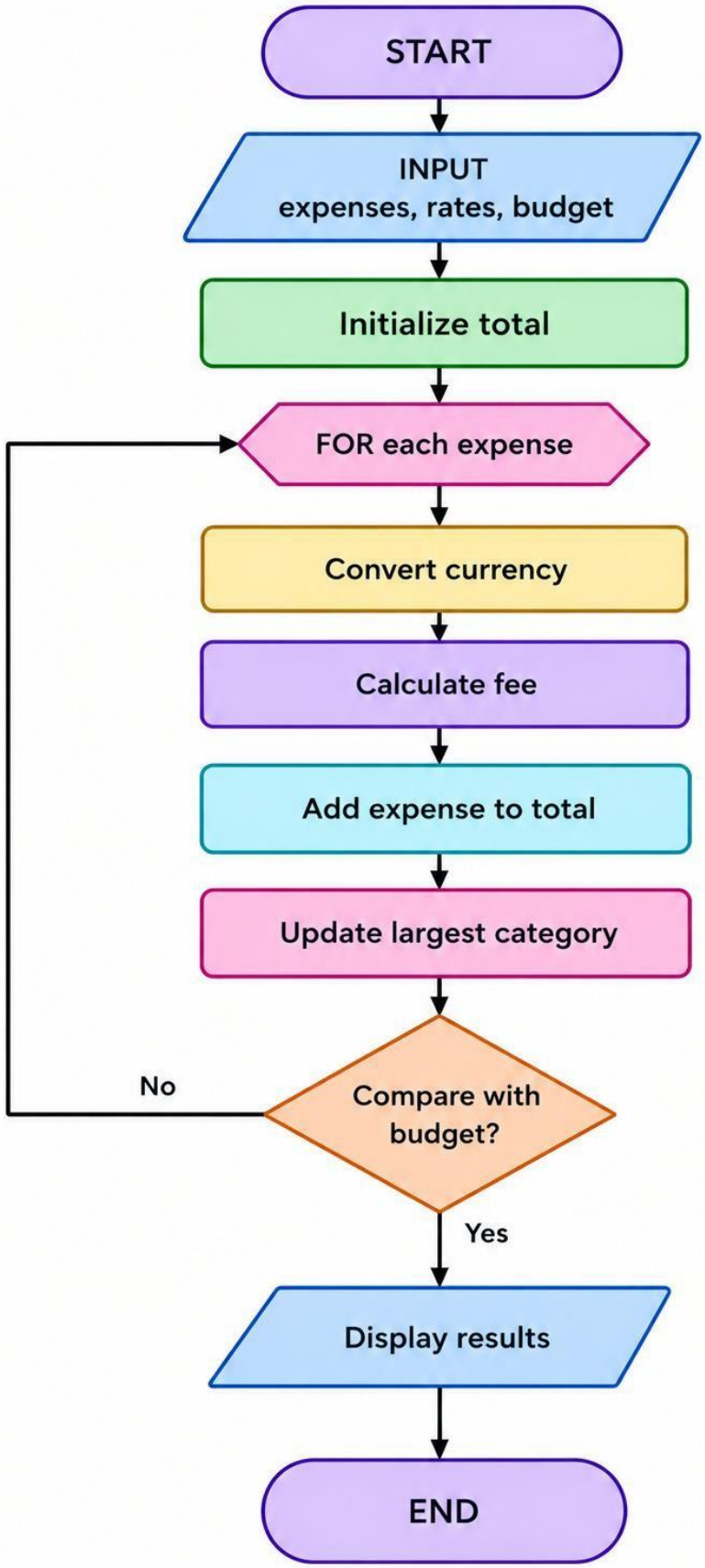
- **UNPACK category, amount, and currency.**
- **CONVERT the amount into the base currency using the exchange rate.**
- **CALCULATE the conversion fee.**
- **CALCULATE the total expense including the fee.**
- **ADD the expense to total spending.**
- **UPDATE the largest expense category if necessary.**

Step 6: COMPARE total spending with the travel budget.

Step 7: DISPLAY total spending, budget status, and largest expense category.

Step 8: END.

4. FLOWCHART


START
INPUT
expenses, rates, budget
Initialize total
FOR each expense
Convert currency
Calculate fee
Add expense to total
Update largest category
Compare with
budget?
No
Yes
Display results
END

5. PSEUDOCODE

```
total_spending = 0
largest_expense = 0
largest_category = ""

FOR expense IN expenses:
    UNPACK category, amount, currency
    converted = amount * exchange_rate[currency]
    fee = converted * fee_percent / 100
    total = converted + fee
    total_spending = total_spending + total

    IF total > largest_expense:
        largest_expense = total
        largest_category = category

IF total_spending <= budget:
    status = "Within Budget"
ELSE:
    status = "Over Budget"

PRINT total_spending, status, largest_category
```

6. VARIABLES

category, amount, currency, exchange_rate, converted, fee_percent, fee, total, total_spending, budget, largest_expense, largest_category, status

7. DATA TYPES

- **float: amount, exchange rate, fee, spending, budget**
- **str: category, currency, status**

- **tuple: expense details**
- **list: expenses**
- **dict: exchange rates**

8. OPERATORS

Arithmetic: +, -, *, /

Relational: >=, <=, <, >

Logical: AND, OR

Assignment: =

9. CONDITIONAL STATEMENTS

if total_spending <= budget: passelse: pass

10. LOOPS

for expense in expenses: pass

11. STRINGS

f"{category} - Expense: ₹{total:.2f}"currency.upper()len(category)

12. PATTERN GENERATION

print("-" * 30)for i in range(1, 4): print("*" * i)

13. LISTS

expenses = []converted_expenses = []converted_expenses.append()

14. NESTED LISTS

expenses = [("Hotel", 100, "USD"), ("Food", 50, "EUR")]

15. TUPLES

("Hotel", 100, "USD") stores one travel expense.

16. TUPLE OPERATIONS

expenses[0]expense[0]len(expenses)

17. UNPACKING

category, amount, currency = expense

INPUT(CODE) :

```
amount = float(input("Enter foreign currency amount: "))
rate = float(input("Enter exchange rate: "))
fee = float(input("Enter conversion fee: "))
budget = float(input("Enter budget in rupees: "))

total = amount * rate + fee

print("Total Travel Expense =", total)

if total <= budget:
    print("Within Budget")
else:
    print("Budget Exceeded")
```

OUTPUT :

```
>>>
=== RESTART: C:/Users/M S I/Desktop/PYTHON-ACTIVITY BASED/program9.py ==
Enter foreign currency amount: 500
Enter exchange rate: 1000
Enter conversion fee: 200
Enter budget in rupees: 300
Total Travel Expense = 500200.0
Budget Exceeded
>>>
```

CASE-10 CAMOUS EVENT FINANCIAL PLANNING

1. PROBLEM STATEMENT

Design a system that calculates campus event expenses, participant contributions, sponsorship shortfalls, and the minimum number of registrations required to break even.

2. COMPUTATIONAL THINKING

- **Decomposition: Expense calculation, sponsorship calculation, participant contribution, break-even analysis.**
- **Pattern Recognition: Apply the same financial calculations to each event.**
- **Abstraction: Consider venue, equipment, catering, transport, certificates, sponsorship, and registration fee.**
- **Algorithm Design: Input expenses → Calculate total cost → Subtract sponsorship → Calculate contributions → Determine break-even registrations.**

3. ALGORITHM

Step 1: START.

Step 2: INPUT event details as a list of tuples (name, venue_cost, equipment_cost, catering_cost, transport_cost, certificate_cost).

Step 3: INPUT sponsorship amount, expected participants, and registration fee.

Step 4: SET results = [].

Step 5: FOR each event:

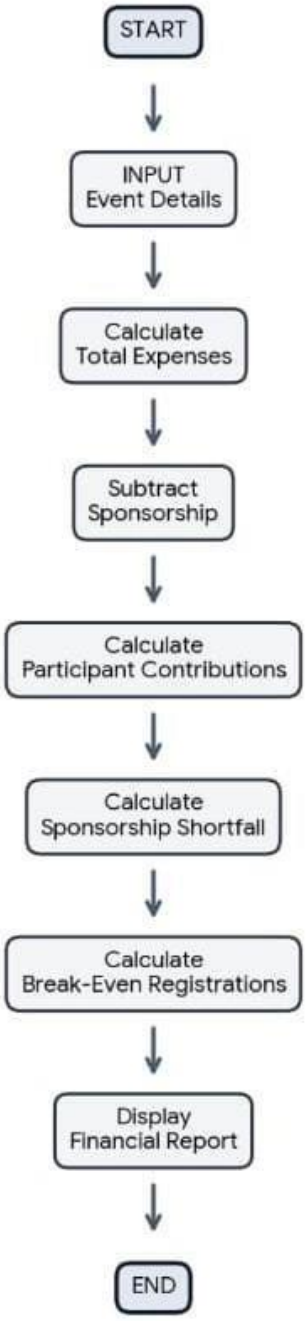
- **UNPACK event name and expenses.**
- **CALCULATE total event cost by adding all expenses.**
- **CALCULATE remaining cost after sponsorship.**
- **CALCULATE participant contributions using registration fee and expected participants.**
- **CALCULATE sponsorship shortfall if sponsorship is insufficient to cover the event cost.**
- **CALCULATE minimum registrations needed to break even.**
- **ADD event details to results.**

Step 6: DISPLAY total expenses, sponsorship shortfall, participant contributions, and break-even registrations.

Step 7: END.

4. FLOWCHART :

Event Budget and Break-Even Financial Flow



5. PSEUDOCODE

results = []

FOR event IN events:

UNPACK name, venue, equipment, catering, transport, certificates

total_cost = venue + equipment + catering + transport + certificates

remaining_cost = MAX(0, total_cost - sponsorship)

contributions = participants * registration_fee

shortfall = MAX(0, total_cost - sponsorship - contributions)

IF registration_fee > 0:

break_even = CEILING(remaining_cost / registration_fee)

ELSE:

break_even = 0

results.append((name, total_cost, contributions, shortfall, break_even))

PRINT results

6. VARIABLES

name, venue, equipment, catering, transport, certificates, total_cost, sponsorship, remaining_cost, participants, registration_fee, contributions, shortfall, break_even, results

7. DATA TYPES

- **int: participants, break-even registrations**
- **float: expenses, sponsorship, registration fee, contributions, shortfall**
- **str: event name**
- **tuple: event details**
- **list: events, results**

8. OPERATORS

Arithmetic: +, -, *, /

Relational: >=, <=, <, >

Logical: AND, OR

Assignment: =

9. CONDITIONAL STATEMENTS

if contributions >= total_cost - sponsorship: passelse: pass

10. LOOPS

for event in events: pass

11. STRINGS

f"{name} - Total Cost: ₹{total_cost:.2f}"name.upper()len(name)

12. PATTERN GENERATION

print("=" * 30)for i in range(1, 4): print("*" * i)

13. LISTS

events = []results = []results.append()

14. NESTED LISTS

events = [("Tech Fest", 10000, 5000, 20000, 5000, 2000), ("Cultural Fest", 8000, 4000, 15000, 3000, 1500)]

15. TUPLES

("Tech Fest", 10000, 5000, 20000, 5000, 2000) stores event expenses.

16. TUPLE OPERATIONS

events[0]event[0]len(events)

17. UNPACKING

name, venue, equipment, catering, transport, certificates = event

INPUT(CODE) :

```
venue = float(input("Enter venue cost: "))
equipment = float(input("Enter equipment cost: "))
food = float(input("Enter food cost: "))
transport = float(input("Enter transport cost: "))
```

```
sponsorship = float(input("Enter sponsorship amount: "))
fee = float(input("Enter registration fee per student: "))

total_cost = venue + equipment + food + transport
shortfall = total_cost - sponsorship

if shortfall <= 0:
    students = 0
else:
    students = int(shortfall / fee)
    if students * fee < shortfall:
        students = students + 1

print("Total Event Cost =", total_cost)
print("Sponsorship =", sponsorship)
print("Minimum Students Required =", students)
```

OUTPUT :

```
>>>
    == RESTART: C:/Users/M S I/Desktop/PYTHON-ACTIVITY BASED/program10.py ==
    Enter venue cost: 1000
    Enter equipment cost: 5000
    Enter food cost: 3000
    Enter transport cost: 2000
    Enter sponsorship amount: 5000
    Enter registration fee per student: 200
    Total Event Cost = 11000.0
    Sponsorship = 5000.0
    Minimum Students Required = 30
>>>
```